

Chapter 5

Functions and Intro to Function Templates

Learning Objectives

- Construct programs modularly from functions.
- Use math library functions and learn about math functions and constants added in recent C++ versions.
- Declare functions with function prototypes.
- View many key C++ standard library headers.
- Use random numbers to implement game-playing apps.
- Declare constants in scoped enums and use constants without their type names via C++20's using enum declarations.
- Use inline functions, references and default arguments.
- Define overloaded functions of the same name that handle various argument types.
- Define function templates that can conveniently generate families of overloaded functions.
- Write and use recursive functions.
- Understand the scope of identifiers.

Introduction

- In this chapter, we introduce building **custom function definitions**.
 - We overview some C++ standard library math functions and constants added in recent C++ versions.
 - We introduce function prototypes and discuss how the compiler uses them, if necessary, to convert the type of an argument in a function call to the type specified in a function's parameter list.
 - We also present an overview of the C++ standard library's headers.
- Next, we demonstrate simulation techniques with **random-number generation**.
 - We simulate a popular casino dice game that uses most of the C++ capabilities we've presented.
 - In the game, we show how to declare constants in scoped enums and discuss C++20's using enum declarations for accessing scoped enum constants directly without their type name.

Introduction.

- We discuss features that help improve program performance, including
 - **inline functions** that can eliminate a function call's overhead and
 - reference parameters for efficiently passing large data items to functions.
- Many applications you develop will have multiple functions with the same name.
 - You'll use this "**function overloading**" technique to implement functions that perform similar tasks for arguments of different types or different numbers of arguments.
 - We introduce **function templates**, which enable you to conveniently define families of overloaded functions.
- We also demonstrate **recursive functions** that call themselves, directly or indirectly, through another function.
- We then present C++'s **scope rules** for determining where identifiers can be referenced in a program.

C++ Program Components

- You typically write C++ programs by combining
 - prepackaged functions and classes available in the C++ standard library,
 - functions and classes available in a vast number of open-source and proprietary third-party libraries, and
 - new functions and classes you and your colleagues write.

C++ Program Components (Cont.)

- The C++ standard library provides a rich collection of functions and classes for
 - math,
 - string processing,
 - regular expressions,
 - input/output,
 - file processing,
 - dates and times,
 - containers (collections of data),
 - algorithms for manipulating the contents of containers,
 - memory management,
 - concurrent programming,
 - and more.

C++ Program Components (Cont.)

- Functions and classes allow you to separate a program's tasks into small self-contained units.
 - So far, each program you've seen has used a combination of C++ standard library features and the main function.
 - In addition, some programs used open-source library features.
 - In this chapter, you'll begin defining custom functions and starting in Chapter 9, you'll define custom classes.

C++ Program Components.

- Some motivations for using functions and classes to create program components include:
 - Software reuse.
 - ◆ For example, in earlier programs, we did not have to define how to create and manipulate strings or read a line of text from the keyboard.
 - ◆ C++ provides these capabilities via the `<string>` header's string class and `getline` function.
 - Avoiding code repetition.
 - Hiding complexity.
 - Easier testing, debugging and maintenance.
- To promote reusability, every function should perform a single, well-defined task, and the function's name should express that task effectively.
 - We'll say lots more about software reusability in our treatment of object-oriented programming.
 - C++20 introduces another construct called modules, which we will discuss in Chapter 16.

Math Library Functions

- In the Objects-Natural case study sections, you've created objects of existing classes, then called their member functions to perform useful tasks.
 - Some reusable code is provided simply as standalone functions, known as [global functions](#).
- The `<cmath>` header provides many global functions (in the `std` namespace) for common mathematical calculations. For example,
`sqrt(900.0)`
calculates the square root of 900.0 and returns the result, 30.0.
 - Function `sqrt` takes a double argument and returns a double result.

Math Library Functions (Cont.)

Function	Description	Example
<code>ceil(x)</code>	rounds x to the smallest integer not less than x	<code>ceil(9.2)</code> is 10.0 <code>ceil(-9.8)</code> is -9.0
<code>cos(x)</code>	trigonometric cosine of x (x in radians)	<code>cos(0.0)</code> is 1.0
<code>exp(x)</code>	exponential function e^x	<code>exp(1.0)</code> is 2.718282 <code>exp(2.0)</code> is 7.389056
<code>fabs(x)</code>	absolute value of x	<code>fabs(5.1)</code> is 5.1 <code>fabs(0.0)</code> is 0.0 <code>fabs(-8.76)</code> is 8.76
<code>floor(x)</code>	rounds x to the largest integer not greater than x	<code>floor(9.2)</code> is 9.0 <code>floor(-9.8)</code> is -10.0
<code>fmod(x, y)</code>	remainder of x/y as a floating-point number	<code>fmod(2.6, 1.2)</code> is 0.2
<code>log(x)</code>	natural logarithm of x (base e)	<code>log(2.718282)</code> is 1.0 <code>log(7.389056)</code> is 2.0
<code>log10(x)</code>	logarithm of x (base 10)	<code>log10(10.0)</code> is 1.0 <code>log10(100.0)</code> is 2.0
<code>pow(x, y)</code>	x raised to power y (x^y)	<code>pow(2, 7)</code> is 128 <code>pow(9, .5)</code> is 3
<code>sin(x)</code>	trigonometric sine of x (x in radians)	<code>sin(0.0)</code> is 0
<code>sqrt(x)</code>	square root of x (where x is a non-negative value)	<code>sqrt(9.0)</code> is 3.0
<code>tan(x)</code>	trigonometric tangent of x (x in radians)	<code>tan(0.0)</code> is 0

Some popular math library functions are summarized in the following table. The variables x and y are of type double.

Math Library Functions (Cont.)

- Additional Math Functions

- Recent C++ versions added dozens of new math functions to the `<cmath>` header.
- Some were entirely new, and some were other versions of existing functions but for arguments of type `float` or `long double` rather than `double`.
 - ◆ The two-argument `hypot` function, for example, calculates a right triangle's hypotenuse. There is also a three-argument version of `hypot` to calculate the hypotenuse in three-dimensional space.
 - ◆ See the complete list <https://en.cppreference.com/w/cpp/numeric/math> and the C++ standard section, "Mathematical functions for floating-point types": <https://timsong-cpp.github.io/cppwp/n4861/c.math>

Math Library Functions (Cont.)

- C++20 New Mathematical Constants and the Header
 - Before C++20, C++ did not provide common mathematical constants, though some C++ implementations defined their own.
 - C++20's new header standardizes the following mathematical constants commonly used in many scientific and engineering applications:

C++ constant	Mathematical expression	C++ constant	Mathematical expression
numbers::e	e	numbers::inv_sqrtpi	$\frac{1}{\sqrt{\pi}}$
numbers::log2e	$\log_2 e$	numbers::sqrt2	$\sqrt{2}$
numbers::log10e	$\log_{10} e$	numbers::sqrt3	$\sqrt{3}$
numbers::ln2	$\log_e(2)$	numbers::inv_sqrt3	$\frac{1}{\sqrt{3}}$
numbers::ln10	$\log_e(10)$	numbers::egamma	Euler-Mascheroni γ constant
numbers::pi	π	numbers::phi	$\frac{(1+\sqrt{5})}{2}$
numbers::inv_pi	$\frac{1}{\pi}$		

Math Library Functions.

- C++17 Mathematical Special Functions
 - C++17 added scores of mathematical special functions for the engineering and scientific communities to the `<cmath>` header.
 - You can see the complete list and brief examples of each on cppreference.com
 - Each function has versions for float, double and long double arguments.

Function Definitions and Function Prototypes

- Let's create a user-defined maximum function that returns the largest of its three int arguments.
- When the next program executes, main reads three integers from the user.
- Then, line 16 calls maximum (defined in lines 20–34).

```
1 // maximum function with a function prototype.
2 #include <iostream>
3
4 using namespace std;
5
6 int maximum(int x, int y, int z); // function prototype
7
8 int main() {
9     cout << "Enter three integer values: ";
10    int int1, int2, int3;
11    cin >> int1 >> int2 >> int3;
12
13    // invoke maximum
14    cout << "The maximum integer value is: "
15         << maximum(int1, int2, int3) << '\n';
16 }
```

```
Enter three integer grades: 86 67 75
The maximum integer value is: 86
```

Function Definitions and Function Prototypes

- In line 33, function maximum returns the largest value back to its caller—in this case, line 16 displays the return value.

```
18 // returns the largest of three integers
19 int maximum(int x, int y, int z) {
20     int maximumValue{x}; // assume x is the largest to start
21
22     // determine whether y is greater than maximumValue
23     if (y > maximumValue) {
24         maximumValue = y; // make y the new maximumValue
25     }
26
27     // determine whether z is greater than maximumValue
28     if (z > maximumValue) {
29         maximumValue = z; // make z the new maximumValue
30     }
31
32     return maximumValue;
33 }
```

Function Definitions and Function Prototypes (Cont.)

- Function maximum

- A **function definition**'s first line (line 19 for function maximum) specifies its **return type**, **function name** and parentheses containing the **parameter list**, which specifies additional information the function needs to perform its task.
 - ◆ The function's first line is known as the function's header.
 - ◆ A parameter list may contain zero or more parameters, each declared with a type and a name.
 - ◆ Two or more parameters are specified using a comma-separated list.
- Function maximum has three int parameters named x, y and z.
- When you call a function, each parameter receives the corresponding argument's value from the function call.

```
18 // returns the largest of three integers
19 □ int maximum(int x, int y, int z) {
```


Function Definitions and Function Prototypes (Cont.)

- Function maximum

- Function maximum first assumes that parameter x has the largest value, so line 20 initializes maximumValue to x's value.

```
18 // returns the largest of three integers
19 int maximum(int x, int y, int z) {
20     int maximumValue{x}; // assume x is the largest to start
21
22     // determine whether y is greater than maximumValue
23     if (y > maximumValue) {
24         maximumValue = y; // make y the new maximumValue
25     }
26
27     // determine whether z is greater than maximumValue
28     if (z > maximumValue) {
29         maximumValue = z; // make z the new maximumValue
30     }
31
32     return maximumValue;
33 }
```

Function Definitions and Function Prototypes (Cont.)

- Function maximum

- Of course, parameter y or z might contain the largest value, so we compare each to maximumValue.
 - ◆ Lines 23–25 determine whether y is greater than maximumValue and, if so, assign y to maximumValue.
 - ◆ Lines 28–30 determine whether z is greater than maximumValue and, if so, assign z to maximumValue.

```
22 // determine whether y is greater than maximumValue
23 if (y > maximumValue) {
24     maximumValue = y; // make y the new maximumValue
25 }
26
27 // determine whether z is greater than maximumValue
28 if (z > maximumValue) {
29     maximumValue = z; // make z the new maximumValue
30 }
```

Function Definitions and Function Prototypes (Cont.)

- Function maximum

- Now, maximumValue contains the largest value, so line 32 returns a copy of that value to the caller.

```
18 // returns the largest of three integers
19 int maximum(int x, int y, int z) {
20     int maximumValue{x}; // assume x is the largest to start
21
22     // determine whether y is greater than maximumValue
23     if (y > maximumValue) {
24         maximumValue = y; // make y the new maximumValue
25     }
26
27     // determine whether z is greater than maximumValue
28     if (z > maximumValue) {
29         maximumValue = z; // make z the new maximumValue
30     }
31
32     return maximumValue;
33 }
```

Function Definitions and Function Prototypes (Cont.)

- Function Prototype for maximum

- You must either define a function before using it or declare it, as in line 6:

```
int maximum(int x, int y, int z); // function prototype
```

- This **function prototype** describes the interface to the maximum function without revealing its implementation.
 - ◆ A function prototype tells the compiler the function's **name**, its **return type** and the **types of its parameters**.
 - ◆ Each prototype ends with a required semicolon (;).
 - ◆ Line 6 indicates that maximum returns an int and requires three int parameters to perform its task.
 - ◆ The types in the function prototype must be the same as those in the corresponding function definition's header (line 19).
 - ◆ The function prototype's parameter names should match those in the function definition, but that's not required.

Math Library Functions (Cont.)

- Parameter Names in Function Prototypes
 - Parameter names in function prototypes are optional (the compiler ignores them),
 - but it's recommended that you use these names for documentation purposes.

Function Definitions and Function Prototypes (Cont.)

- What the Compiler Does with maximum's Function Prototype
 - When compiling the program, the compiler uses the prototype to:
 - ◆ Check that maximum's header (line 19) **matches its prototype** (line 6).
 - ◆ Check that the call to maximum (line 15) contains the **correct number and types of arguments**, and that the arguments' types are in the **correct order**.
 - In this case, all the arguments are of type int.
 - ◆ Check that the **value returned** by the function can be used **correctly in the expression** that called the function.
 - For example, a function declared with the void return type does not return a value, so it cannot be called where a value is expected, such as on the right side of an assignment or in an output statement.
 - ◆ Check that each **argument is consistent with the corresponding parameter's type**
 - For example, a parameter of type double can receive values like 7.35, 22 but not a string like "hello". If the arguments passed to a function do not match the types specified in the function's prototype, the compiler attempts to convert the arguments to those types.
 - Compilation errors occur if the function prototype, function header and function calls do not agree in the return type and number, type and order of arguments and parameters.

Function Definitions and Function Prototypes.

- Returning Control from a Function to Its Caller
 - When a program calls a function, the function performs its task, then **returns control** (and possibly a value) to the point where the function was called.
 - In a function that does not return a result (that is, it has a void return type), control returns to the caller when the program reaches the **function-ending right brace**.
 - A void-return-type function can explicitly return control (and no result) to the caller by executing `return;` anywhere in the function's body.

Order of Evaluation of a Function's Arguments

- The commas in line 15 that separate function maximum's arguments are not comma operators.

```
15 | << maximum(int1, int2, int3) << '\n';
```

- The comma operator guarantees that its operands evaluate left to right.
- However, the **order of evaluation** of a function's arguments is **NOT** specified by the C++ standard.
- Thus, different compilers can evaluate function arguments in different orders.

Order of Evaluation of a Function's Arguments

- Sometimes when a function's arguments are expressions, such as those with calls to other functions, the order in which the compiler evaluates the arguments could affect the values of one or more of the arguments.
 - If the evaluation order varies among compilers, the argument values passed to the function could vary, causing subtle logic errors.
- If you have doubts about the order of evaluation of a function's arguments and whether the order would affect the values passed to the function,
 - assign the arguments to variables before the call,
 - then pass those variables as arguments to the function.

Function-Prototype and Argument-Coercion Notes

- A function prototype is required unless the function is defined before it's used.
 - If a function is defined before it's called, its definition also serves as the function's prototype, so a separate prototype is unnecessary.
 - A compilation error occurs if a function is called before it's defined and that function does not have a function prototype.
 - When you use a standard library function like `sqrt`, you do not have access to its definition, so it cannot be defined in your code before you call the function.
 - Instead, you must include the header (in this case, `<cmath>`) containing the function's prototype.
- Though it's possible to omit function prototypes when functions are defined before they're used, providing them avoids tying your code to the order in which functions are defined, which can change as a program evolves.

Function-Prototype and Argument-Coercion Notes (Cont.)

- Function Signatures and Function Prototypes
 - A function's name and its parameter types together are known as the **function signature** or simply the signature.
 - ◆ The function's return type is not part of the function signature.
 - ◆ A function's scope is the region of a program in which the function is known and accessible. Functions in the same scope must have unique signatures.
 - In the program, if the function prototype in line 6 had been written `void maximum(int x, int y, int z);` the compiler would report an error because the prototype's void return type would differ from the function header's int return type.
 - Similarly, such a prototype would cause the statement `cout << maximum(6, 7, 0);` to generate a compilation error because that statement depends on maximum returning a value to be displayed.
 - Function prototypes help you find many errors at **compile-time**, which is always better than finding them at runtime.

Function-Prototype and Argument-Coercion Notes (Cont.)

- Argument Coercion

- An important feature of function prototypes is **argument coercion**—forcing arguments to the types specified by the parameter declarations.
 - ◆ For example, a program can call a function with an integer argument even though the function prototype specifies a double parameter.
 - ◆ The function will still work correctly, provided this is not a narrowing conversion.
 - ◆ A compilation error occurs if the arguments in a function call **cannot** be **implicitly converted** to the expected types specified in the function's prototype.

Function-Prototype and Argument-Coercion Notes (Cont.)

- Argument-Promotion Rules and Implicit Conversions
 - Sometimes, argument values that do not correspond precisely to the parameter types in the function prototype can be converted by the compiler to the proper type before the function is called.
 - These conversions occur as specified by C++'s [promotion rules](#), which indicate the implicit conversions allowed between fundamental types.
 - ◆ An int can be converted to a double.
 - ◆ A double also can be converted to an int, but this narrowing conversion truncates the double's fractional part.
 - ◆ Recall that doubles can hold numbers of much greater magnitude than ints, so the loss of data in a narrowing conversion can be considerable.

Function-Prototype and Argument-Coercion Notes (Cont.)

- Argument-Promotion Rules and Implicit Conversions
 - Values also might be modified when converting
 - ◆ large integer types to small integer types (e.g., long to short),
 - ◆ signed to unsigned types, or
 - ◆ unsigned to signed types.
 - Variables of unsigned integer types can represent values from 0 to approximately twice the positive range of the corresponding signed integer types.
 - The C++ Core Guidelines indicate that unsigned types should not be used to ensure or document that a value is non-negative.

Function-Prototype and Argument-Coercion Notes (Cont.)

- Argument-Promotion Rules and Implicit Conversions
 - The promotion rules also apply to mixed-type expressions containing values of two or more data types.
 - Each value's type is promoted to the expression's "highest" type.
 - The promotion uses a temporary copy of each value—the original values remain unchanged.

Function-Prototype and Argument-Coercion Notes (Cont.)

- Argument-Promotion Rules and Implicit Conversions
 - The following list shows the arithmetic types in order from “highest” to “lowest” type:
 - ◆ long double
 - ◆ double
 - ◆ float
 - ◆ unsigned long long int
 - ◆ long long int
 - ◆ unsigned long int
 - ◆ long int
 - ◆ unsigned int
 - ◆ int
 - ◆ unsigned short int
 - ◆ short int
 - ◆ unsigned char
 - ◆ char and signed char
 - ◆ bool

Function-Prototype and Argument-Coercion Notes (Cont.)

- Conversions Can Result in Incorrect Values
 - Converting values to lower types can cause **narrowing conversion errors** or warnings.
 - ◆ If you pass a double argument to a square function with an int parameter, the argument is converted to int (a lower type and thus a narrowing conversion).
 - ◆ In this case, square could return an incorrect value. For example, square(4.5) would return 16, not 20.25.
 - ◆ Some compilers warn you about this.

Function-Prototype and Argument-Coercion Notes (Cont.)

- Narrowing Conversions with the Guidelines Support Library
 - If you must perform an explicit narrowing conversion, the C++ Core Guidelines recommend using `narrow_cast` from the Guidelines Support Library (GSL).
 - This library has several implementations.
 - Microsoft's open-source version has been tested on numerous platform/compiler combinations.

Function-Prototype and Argument-Coercion Notes.

- Narrowing Conversions with the Guidelines Support Library
 - You can download Microsoft's open-source GSL from <https://github.com/Microsoft/GSL>
 - The GSL is a **header-only library**, so you can use it in your programs simply by including the header `<gsl/gsl>`.
 - ◆ You must point your compiler to the GSL folder's include subfolder so the compiler knows where to find the header file, as you did when you used the Boost Multiprecision class `cpp_int`.
 - The following statement uses a **narrow_cast** (from namespace `gsl`) to convert the double value 7.5 to the int value 7:
`gsl::narrow_cast<int>(7.5)`
The value in parentheses is converted to the type in angle brackets, `<>`.

C++ Standard Library Headers

- The C++ standard library is divided into many headers containing function prototypes and sometimes full definitions for the functions in that header.
 - The headers also contain definitions of various class types, functions and constants needed by those functions.
 - A header “instructs” the compiler on how to interface with library and user-written components.
- The following table lists some common C++ standard library headers, many of which are discussed later in this course.
 - For a complete list of C++20 standard library headers, visit <https://en.cppreference.com/w/cpp/header>

C++ Standard Library Headers (Cont.)

Standard library header	Explanation
<code><iostream></code>	Classes for the C++ standard input and output capabilities, introduced in Chapter 2 and covered in more detail in Chapter 19, Stream I/O and C++20 Text Formatting .
<code><iomanip></code>	Functions for stream manipulators that format streams of data. This header is first used in Section 3.9 and is discussed in more detail in Chapter 19.
<code><cmath></code>	Math library functions (Section 5.3).
<code><cstdlib></code>	Functions for converting numbers to text, converting text to numbers, memory allocation, random numbers and more. Portions of the header are covered in Chapter 11, Operator Overloading, Copy/Move Semantics and Smart Pointers , and Chapter 12, Exceptions and a Look Forward to Contracts .
<code><random></code>	Random-number generation capabilities (discussed in this chapter and used in subsequent chapters).
<code><ctime></code> , <code><chrono></code>	Functions and types for manipulating the time and date. <code><chrono></code> was enhanced with many more features in C++20. We use several <code><chrono></code> timing features in Chapter 17, Parallel Algorithms and Concurrency: A High-Level View .

C++ Standard Library Headers (Cont.)

<code><array></code> , <code><vector></code> , <code><list></code> , <code><tuple></code> , <code><forward_list></code> , <code><deque></code> , <code><queue></code> , <code><stack></code> , <code><map></code> , <code><unordered_map></code> , <code><unordered_set></code> , <code><set></code> , <code><bitset></code>	These headers contain classes that implement the C++ standard library's implementations of commonly used data structures—known as containers. The <code><array></code> and <code><vector></code> headers are first introduced in Chapter 6 , arrays, vectors, Ranges and Functional-Style Programming. We discuss all these headers in Chapter 13, Data Structures: Standard Library Containers and Iterators .
<code><cctype></code>	Functions that test characters for specific properties (such as whether the character is a digit or punctuation), and function prototypes for functions that can convert lowercase letters to uppercase letters and vice versa.
<code><cstring></code>	C-style string-processing functions.
<code><typeinfo></code>	Classes for runtime type identification (determining data types at execution time).
<code><exception></code> , <code><stdexcept></code>	Classes for exception handling (discussed in Chapter 12).
<code><memory></code>	Classes and functions for managing memory allocation. This header is used in Chapter 12 .
<code><fstream></code>	Classes that perform input from and output to files on disk (discussed in Chapter 8 , strings, string_views, Text Files, CSV Files and Regex).
<code><string></code>	Definition of class <code>string</code> and functions <code>getline</code> and <code>to_string</code> (discussed in Chapter 8).
<code><sstream></code>	Classes that perform input from strings in memory and output to strings in memory (discussed in Chapter 8).

C++ Standard Library Headers (Cont.)

<code><array></code> , <code><vector></code> , <code><list></code> , <code><tuple></code> , <code><forward_list></code> , <code><deque></code> , <code><queue></code> , <code><stack></code> , <code><map></code> , <code><unordered_map></code> , <code><unordered_set></code> , <code><set></code> , <code><bitset></code>	These headers contain classes that implement the C++ standard library's implementations of commonly used data structures—known as containers. The <code><array></code> and <code><vector></code> headers are first introduced in Chapter 6 , arrays, vectors, Ranges and Functional-Style Programming. We discuss all these headers in Chapter 13, Data Structures: Standard Library Containers and Iterators .
<code><cctype></code>	Functions that test characters for specific properties (such as whether the character is a digit or punctuation), and function prototypes for functions that can convert lowercase letters to uppercase letters and vice versa.
<code><cstring></code>	C-style string-processing functions.
<code><typeinfo></code>	Classes for runtime type identification (determining data types at execution time).
<code><exception></code> , <code><stdexcept></code>	Classes for exception handling (discussed in Chapter 12).
<code><memory></code>	Classes and functions for managing memory allocation. This header is used in Chapter 12 .
<code><fstream></code>	Classes that perform input from and output to files on disk (discussed in Chapter 8 , strings, string_views, Text Files, CSV Files and Regex).
<code><string></code>	Definition of class <code>string</code> and functions <code>getline</code> and <code>to_string</code> (discussed in Chapter 8).
<code><sstream></code>	Classes that perform input from strings in memory and output to strings in memory (discussed in Chapter 8).

C++ Standard Library Headers (Cont.)

<code><functional></code>	Classes and functions used by C++ standard library algorithms. This header is used in Chapter 14, Standard Library Algorithms and C++20 Ranges & Views .
<code><iterator></code>	Classes for accessing data in the C++ standard library containers. This header is used in Chapter 13, Data Structures: Standard Library Containers and Iterators .
<code><algorithm></code>	Functions for manipulating data in C++ standard library containers. This header is used in Chapter 13 .
<code><cassert></code>	Macros for adding diagnostics that aid program debugging. This header is used in the Preprocessor appendix at https://deitel.com/cpphtp11 .
<code><cfloat></code>	C-style floating-point size limits of the system. C++ programs typically use the <code><limits></code> header instead.
<code><climits></code>	C-style integral size limits of the system. C++ programs typically use the <code><limits></code> header instead.
<code><cstdio></code>	C-style standard input/output library functions.
<code><locale></code>	Classes and functions for processing data in the natural form for different languages (e.g., monetary formats, sorting strings, character presentation).
<code><limits></code>	Classes for defining the numerical data type limits on each computer platform – this is C++'s version of <code><climits></code> and <code><cfloat></code> .
<code><utility></code>	Classes and functions that are used by many C++ standard library headers.

C++ Standard Library Headers (Cont.)

<code><thread></code> , <code><mutex></code> , <code><shared_mutex></code> , <code><future></code> , <code><condition_variable></code>	Capabilities for multithreaded application development that enable applications to take advantage of multi-core processors (discussed in Chapter 17, Parallel Algorithms and Concurrency: A High-Level View).
<i>Some Key C++17 New Headers</i>	
<code><any></code>	A class for holding a value of any copyable type.
<code><optional></code>	A template to represent an object that may or may not have a value (discussed in Chapter 13).
<code><variant></code>	Features used to create and manipulate objects of a specified set of types (discussed in Chapter 10, OOP: Inheritance and Runtime Polymorphism).
<code><execution></code>	Features used with the Standard Template Library's parallel algorithms (discussed in Chapter 17).
<code><filesystem></code>	Capabilities for interacting with the local file system's files and folders.
<i>Some Key C++20 New Headers</i>	
<code><concepts></code>	Capabilities for constraining the types used with templates (discussed in Chapter 15, Templates, C++20 Concepts and Metaprogramming).
<code><coroutine></code>	Capabilities for asynchronous programming with coroutines (discussed in Chapter 17).
<code><compare></code>	Support for the new three-way comparison operator <code><=></code> (discussed in Chapter 11).

C++ Standard Library Headers (Cont.)

<code><thread></code> , <code><mutex></code> , <code><shared_mutex></code> , <code><future></code> , <code><condition_variable></code>	Capabilities for multithreaded application development that enable applications to take advantage of multi-core processors (discussed in Chapter 17, Parallel Algorithms and Concurrency: A High-Level View).
<i>Some Key C++17 New Headers</i>	
<code><any></code>	A class for holding a value of any copyable type.
<code><optional></code>	A template to represent an object that may or may not have a value (discussed in Chapter 13).
<code><variant></code>	Features used to create and manipulate objects of a specified set of types (discussed in Chapter 10, OOP: Inheritance and Runtime Polymorphism).
<code><execution></code>	Features used with the Standard Template Library's parallel algorithms (discussed in Chapter 17).
<code><filesystem></code>	Capabilities for interacting with the local file system's files and folders.
<i>Some Key C++20 New Headers</i>	
<code><concepts></code>	Capabilities for constraining the types used with templates (discussed in Chapter 15, Templates, C++20 Concepts and Metaprogramming).
<code><coroutine></code>	Capabilities for asynchronous programming with coroutines (discussed in Chapter 17).
<code><compare></code>	Support for the new three-way comparison operator <code><=></code> (discussed in Chapter 11).

C++ Standard Library Headers.

<i>Some Key C++20 New Headers</i>	
<code><concepts></code>	Capabilities for constraining the types used with templates (discussed in Chapter 15, Templates, C++20 Concepts and Metaprogramming).
<code><coroutine></code>	Capabilities for asynchronous programming with coroutines (discussed in Chapter 17).
<code><compare></code>	Support for the new three-way comparison operator <code><=></code> (discussed in Chapter 11).
<code><format></code>	New concise and powerful text-formatting capabilities (discussed throughout the book).
<code><ranges></code>	Capabilities that support functional-style programming (discussed in Chapters 6 and 13).
<code></code>	Capabilities for creating views into contiguous sequences of objects (discussed in Chapter 17).
<code><bit></code>	Standardized bit-manipulation operations.
<code><stop_token></code> , <code><semaphore></code> , <code><latch></code> , <code><barrier></code>	Additional capabilities that support multithreading (discussed in Chapter 17).

Case Study: Random-Number Generation

- We now take a brief and hopefully entertaining diversion into a popular programming application—simulation and game playing.
- In this section and the next, we develop a game-playing program that includes multiple functions.

Case Study: Random-Number Generation (Cont.)

- Header `<random>`

- The element of chance can be introduced into your applications with features from the header `<random>`.
- These features replace the **deprecated rand function**, which was brought into C++ from the C standard library.
 - ◆ The rand function does not have “good statistical properties” and can be predictable.
 - ◆ This makes programs using rand **less secure**.
- The more secure features in `<random>` can produce nondeterministic random numbers—a set of random numbers that can’t be predicted.
 - ◆ Such random-number generators are used in simulations and security scenarios where predictability is undesirable.

Case Study: Random-Number Generation (Cont.)

- Header <random>

- Random-number generation is a sophisticated topic for which mathematicians have developed many algorithms with different statistical properties.
- For flexibility based on how random numbers are used in programs, there are many classes that represent various **random-number generation engines** and distributions:
 - ◆ An **engine** implements a **random-number generation algorithm** that produces random numbers.
 - ◆ A **distribution** controls the **range of values** an engine produces, the value's types (e.g., int, double, etc.) and the value's **statistical properties**.

Case Study: Random-Number Generation (Cont.)

- Header `<random>`

- We'll use the default random-number generation engine—`default_random_engine`—and a `uniform_int_distribution`, which evenly distributes random integers over a specified range.
 - ◆ The default range is from 0 to the maximum int value on your platform.
 - ◆ If the `default_random_engine` and `uniform_int_distribution` truly produce integers at random, every number between 0 and the maximum int value has an equal chance (or probability) of being chosen each time the program requests a random number.

Case Study: Random-Number Generation (Cont.)

- Rolling a Six-Sided Die

- The next program simulates and displays ten random rolls of a six-sided die.
- Line 8 creates a `default_random_engine` object named `engine` to produce random numbers.
- Line 11 initializes the `uniform_int_distribution` object `randomDie` with `{1, 6}`, which indicates that it produces `int` values in the range 1 to 6.

```
1 // Producing random integers in the range 1 through 6.
2 #include <iostream>
3 #include <random> // contains random-number generation features
4 using namespace std;
5
6 int main() {
7     // engine that produces random numbers
8     default_random_engine engine{};
9
10    // distribution that produces the int values 1-6 with equal likelihood
11    uniform_int_distribution randomDie{1, 6};
```


Case Study: Random-Number Generation (Cont.)

- Rolling a Six-Sided Die

- The expression

`randomDie(engine)`

in line 15 returns one random int in the range 1 to 6.

```
1 // Producing random integers in the range 1 through 6.
2 #include <iostream>
3 #include <random> // contains random-number generation features
4 using namespace std;
5
6 int main() {
7     // engine that produces random numbers
8     default_random_engine engine{};
9
10    // distribution that produces the int values 1-6 with equal likelihood
11    uniform_int_distribution randomDie{1, 6};
12
13    // display 10 random die rolls
14    for (int counter{1}; counter <= 10; ++counter) {
15        cout << randomDie(engine) << " ";
16    }
17
18    cout << '\n';
19 }
```

3 1 3 6 5 2 6 6 1 2

Case Study: Random-Number Generation (Cont.)

- Rolling a Six-Sided Die 60,000,000 Times
 - To show that the random values produced in the preceding program occur with approximately equal likelihood, the program simulates 60,000,000 rolls of a die.
 - Each int in the range 1 to 6 should appear approximately 10,000,000 times (one-sixth of the rolls).

```
1 // Rolling a six-sided die randomly 60,000,000 times.
2 #include <format>
3 #include <iostream>
4 #include <random>
5 using namespace std;
6
7 int main() {
8     // set up random-number generation
9     default_random_engine engine{};
10    uniform_int_distribution randomDie{1, 6};
11
12    int frequency1{0}; // count of 1s rolled
13    int frequency2{0}; // count of 2s rolled
14    int frequency3{0}; // count of 3s rolled
15    int frequency4{0}; // count of 4s rolled
16    int frequency5{0}; // count of 5s rolled
17    int frequency6{0}; // count of 6s rolled
```

Case Study: Random-Number Generation (Cont.)

● Digit Separators

- Typing a number like 60,000,000 as 60000000 in a C++ program can be error-prone.
- You can make such numbers more readable and reduce errors by using the **digit separator** ' (a single-quote character), which you insert between groups of digits, as in line 20.

```
19 // summarize results of 60,000,000 rolls of a die
20 for (int roll = 1; roll <= 60'000'000; ++roll) {
21     // determine roll value 1-6 and increment appropriate counter
22     switch (roll % 6 + 1) {
23         case 1:
24             ++frequency1; // increment the 1s counter
25             break;
26         case 2:
27             ++frequency2; // increment the 2s counter
28             break;
29         case 3:
30             ++frequency3; // increment the 3s counter
31             break;
32         case 4:
33             ++frequency4; // increment the 4s counter
34             break;
35         case 5:
36             ++frequency5; // increment the 5s counter
37             break;
38         case 6:
39             ++frequency6; // increment the 6s counter
40             break;
41         default: // invalid value
42             cout << "Program should never get here!";
43             break;
44     }
45 }
```

Case Study: Random-Number Generation (Cont.)

- Summarizing the Frequencies

- The `face` variable's definition in the `switch`'s initializer (line 22) is preceded by `const`.

```
22  switch (const int face{randomDie(engine)}) {
```

- ◆ This is a good practice for any variable that should not change once initialized.
- ◆ This enables the compiler to report errors if you accidentally attempt to modify the variable.

Case Study: Random-Number Generation (Cont.)

- Summarizing the Frequencies

- The `switch`'s default case (lines 41–43) should never execute because the controlling expression (`face`) always has values from 1 through 6.
 - ◆ Many programmers provide a default case in every `switch` statement to catch errors, even if they feel confident that their programs are error-free.
- After introducing arrays in Chapter 6, we show how to elegantly replace the entire `switch` with a single-line statement.

```
41         default: // invalid value
42             cout << "Program should never get here!";
43             break;
44     }
45 }
```

Case Study: Random-Number Generation (Cont.)

- Summarizing the Frequencies.
 - Possible simulation results could be as follows:

```
47     cout << format("{:>4}{:>13}\n", "Face", "Frequency");
48     cout << format("{:>4d}{:>13d}\n", 1, frequency1)
49         << format("{:>4d}{:>13d}\n", 2, frequency2)
50         << format("{:>4d}{:>13d}\n", 3, frequency3)
51         << format("{:>4d}{:>13d}\n", 4, frequency4)
52         << format("{:>4d}{:>13d}\n", 5, frequency5)
53         << format("{:>4d}{:>13d}\n", 6, frequency6);
54 }
```

Face	Frequency
1	9997896
2	10000608
3	9996800
4	10000729
5	10003444
6	10000523

Case Study: Random-Number Generation (Cont.)

- Seeding the Random-Number Generator
 - Executing the program of random die again produces the same sequence of values. How can these be random numbers?
 - ◆ The default `random_engine` actually generates **pseudorandom numbers**. Repeatedly executing the programs produces sequences of numbers that appear to be random.
 - ◆ However, the sequences actually **repeat** themselves each time these programs execute.
 - When debugging a simulation, random-number repeatability is essential **for proving that corrections** to the program work properly.
 - Once you've thoroughly debugged your simulation, you can condition it to produce a different sequence of random numbers for each execution.
 - ◆ This is called randomizing.
 - ◆ You initialize the default `random_engine` with an unsigned int argument that seeds the default `random_engine` to produce a different sequence of random numbers.

Case Study: Random-Number Generation (Cont.)

- Seeding the default_random_engine
 - The next program demonstrates seeding the default_random_engine (line 14) with an unsigned int that you input (line 11).

```
1 // Randomizing the die-rolling program.
2 #include <iostream>
3 #include <random>
4
5 using namespace std;
6
7 int main() {
8     unsigned int seed{0}; // stores the seed entered by the user
9
10    cout << "Enter seed: ";
11    cin >> seed;
12
13    // set up random-number generation
14    default_random_engine engine{seed}; // seed the engine
15    uniform_int_distribution randomDie{1, 6};
16
17    // display 10 random die rolls
18    for (int counter{1}; counter <= 10; ++counter) {
19        cout << randomDie(engine) << " ";
20    }
21
22    cout << '\n';
23 }
```

Enter seed: 67
6 2 5 6 6 2 2 6 2 1

Enter seed: 432
3 5 6 1 6 1 4 4 2 2

Enter seed: 67
6 2 5 6 6 2 2 6 2 1

Case Study: Random-Number Generation (Cont.)

- Seeding the default_random_engine
 - The program produces a different random-number sequence each time it executes, provided that you enter a different seed.
 - We used the same seed in the first and third executions, so the same series of 10 numbers is displayed in each.

```
4
5     using namespace std;
6
7     int main() {
8         unsigned int seed{0}; // stores the seed entered by the user
9
10        cout << "Enter seed: ";
11        cin >> seed;
12
13        // set up random-number generation
14        default_random_engine engine{seed}; // seed the engine
15        uniform_int_distribution randomDie{1, 6};
16
17        // display 10 random die rolls
18        for (int counter{1}; counter <= 10; ++counter) {
19            cout << randomDie(engine) << " ";
20        }
21
22        cout << '\n';
23    }
```

Enter seed: 67
6 2 5 6 6 2 2 6 2 1

Enter seed: 432
3 5 6 1 6 1 4 4 2 2

Enter seed: 67
6 2 5 6 6 2 2 6 2 1

Case Study: Random-Number Generation (Cont.)

- Seeding the default_random_engine
 - For security, you must ensure that your program seeds the random-number generator differently (and only once) each time the program executes; otherwise, an attacker could determine the sequence of pseudorandom numbers that would be produced.

Case Study: Random-Number Generation.

- Seeding the Random-Number Generator with `random_device`
 - You'll generally seed the random-number generator engine with a `random_device` object (from header `<random>`).
 - ◆ A `random_device` produces evenly spread random integers, which cannot be predicted.
 - However, the `random_device` documentation indicates that for performance reasons, it's typically used only to seed random number generation engines.
 - ◆ Also, `random_device` might be predictable on some platforms.
 - ◆ So, check your compiler's documentation before relying on this for secure applications.

Case Study: Game of Chance; Introducing Scoped enums

- One of the most popular games of chance is a dice game known as “craps,” which is played in casinos and back alleys worldwide. The rules of the game are straightforward:

A player rolls two dice. Each die has six faces containing 1, 2, 3, 4, 5 and 6 spots. After the dice have come to rest, the sum of the spots on the two upward faces is calculated. If the sum is 7 or 11 on the first roll, the player wins. If the sum is 2, 3 or 12 on the first roll (called “craps”), the player loses (i.e., the “house” wins). If the sum is 4, 5, 6, 8, 9 or 10 on the first roll, then that sum becomes the player’s “point.” To win, you must keep rolling the dice until you “make your point.” The player loses by rolling a 7 before making the point.

Case Study: Game of Chance; Introducing Scoped enums (Cont.)

- In the rules, notice that the player must roll two dice on the first and subsequent rolls.
- We will define a rollDice function to roll the dice and compute and display their sum.
 - The function may be called multiple times—once for the game's first roll and possibly many more times if the player does not win or lose on the first roll.
- Below are the outputs of several sample executions showing:
 - winning on the first roll by rolling a 7,
 - winning on a subsequent roll by “making the point” before rolling a 7,
 - losing on the first roll by rolling a 12, and
 - losing on a subsequent roll by rolling a 7 before “making the point.”

Case Study: Game of Chance; Introducing Scoped enums (Cont.)

- Implementing the Game

- The craps program simulates the game using two functions—`main` and `rollDice`—and the `switch`, `while`, `if...else` and nested `if...else` statements.
- Function `rollDice`'s prototype (line 7) indicates that the function takes no arguments (empty parentheses) and returns an `int` (the sum of the dice).

```
1 // Craps simulation.
2 #include <format>
3 #include <iostream>
4 #include <random>
5 using namespace std;
6
7 int rollDice(); // rolls dice, calculates and displays sum
```

Case Study: Game of Chance; Introducing Scoped enums (Cont.)

- Scoped enums

- The player may win or lose on the first or any subsequent roll.
 - ◆ Line 11 declares a userdefined type called a scoped enumeration that is introduced by the keywords `enum class`, followed by a type name (`Status`) and a set of identifiers representing integer constants.
 - ◆ Line 15 declares variable `gameStatus` as the new type `Status`.
 - This variable can store only values represented by the identifiers in line 11's braces.

```
9  int main() {  
10     // scoped enumeration with constants that represent the game status  
11     enum class Status {keepRolling, won, lost};  
12  
13     int myPoint{0}; // point if no win or loss on first roll  
14     Status gameStatus{Status::keepRolling}; // game is not over  
15 }
```

Case Study: Game of Chance; Introducing Scoped enums (Cont.)

- Scoped `enums`

- The underlying values of these enumeration constants are of type `int`.
- They start at 0 and increment by 1 by default.
 - ◆ In the `Status` enumeration, the constant `keepRolling` has the value 0, `won` has the value 1, and `lost` has the value 2.
 - ◆ The identifiers in an enum class must be unique.
 - ◆ Variables of user-defined type `Status` can be assigned only the constants declared in the enumeration.

```
9  int main() {  
10     // scoped enumeration with constants that represent the game status  
11     enum class Status {keepRolling, won, lost};  
12  
13     int myPoint{0}; // point if no win or loss on first roll  
14     Status gameStatus{Status::keepRolling}; // game is not over  
15 }
```


Case Study: Game of Chance; Introducing Scoped enums (Cont.)

- Scoped enums

- By convention, you should capitalize the first letter of an enum class's name and the first letter of each subsequent word in a multi-word enum class name (e.g., ProductCode).
- The C++ Core Guidelines state that [constants in an enum class should use the same naming conventions as variables](#).

```
9  int main() {  
10     // scoped enumeration with constants that represent the game status  
11     enum class Status {keepRolling, won, lost};  
12  
13     int myPoint{0}; // point if no win or loss on first roll  
14     Status gameStatus{Status::keepRolling}; // game is not over  
15 }
```

Case Study: Game of Chance; Introducing Scoped enums (Cont.)

- Scoped enums

- To reference a scoped enum constant, qualify the constant with the scoped enum's type name (i.e., Status) and the [scope-resolution operator \(::\)](#), as shown in line 14, which initializes gameStatus to Status::keepRolling.
 - ◆ For a win, the program sets gameStatus to Status::won.
 - ◆ For a loss, the program sets gameStatus to Status::lost.

```
9  int main() {  
10     // scoped enumeration with constants that represent the game status  
11     enum class Status {keepRolling, won, lost};  
12  
13     int myPoint{0}; // point if no win or loss on first roll  
14     Status gameStatus{Status::keepRolling}; // game is not over  
15 }
```

Case Study: Game of Chance; Introducing Scoped enums (Cont.)

- Winning or Losing on the First Roll
 - The switch's initializer (line 17) creates the variable `sumOfDice` and initializes it by calling `rollDice`.
 - ◆ If the roll is 7 or 11, line 20 sets `gameStatus` to `Status::won`.
 - ◆ If the roll is 2, 3, or 12, line 25 sets `gameStatus` to `Status::lost`.

```
13     int myPoint{0}; // point if no win or loss on first roll
14     Status gameStatus{Status::keepRolling}; // game is not over
15
16     // determine game status and point (if needed) based on first roll
17     switch (const int sumOfDice{rollDice()}) {
18         case 7: // win with 7 on first roll
19         case 11: // win with 11 on first roll
20             gameStatus = Status::won;
21             break;
22         case 2: // lose with 2 on first roll
23         case 3: // lose with 3 on first roll
24         case 12: // lose with 12 on first roll
25             gameStatus = Status::lost;
26             break;
27         default: // did not win or lose, so remember point
28             myPoint = sumOfDice; // remember the point
29             cout << format("Point is {}\n", myPoint);
30             break; // optional (but recommended) at end of switch
31     }
```

Case Study: Game of Chance; Introducing Scoped enums (Cont.)

- Winning or Losing on the First Roll

- For other values,

- ◆ gameStatus remains unchanged (Status::keepRolling, which was initially set in line 14),
 - ◆ line 28 saves sumOfDice in myPoint, and
 - ◆ line 29 displays myPoint.

```
13     int myPoint{0}; // point if no win or loss on first roll
14     Status gameStatus{Status::keepRolling}; // game is not over
15
16     // determine game status and point (if needed) based on first roll
17     switch (const int sumOfDice{rollDice()}) {
18         case 7: // win with 7 on first roll
19             case 11: // win with 11 on first roll
20                 gameStatus = Status::won;
21                 break;
22             case 2: // lose with 2 on first roll
23             case 3: // lose with 3 on first roll
24             case 12: // lose with 12 on first roll
25                 gameStatus = Status::lost;
26                 break;
27         default: // did not win or lose, so remember point
28             myPoint = sumOfDice; // remember the point
29             cout << format("Point is {}\n", myPoint);
30             break; // optional (but recommended) at end of switch
31     }
```

Case Study: Game of Chance; Introducing Scoped enums (Cont.)

- Continuing to Roll

- After the first roll, if gameStatus is Status::keepRolling, execution proceeds with the \ while statement.
 - ◆ In each loop iteration, the if statement's initializer (line 36) calls rollDice to produce a new sumOfDice.
 - ◆ If sumOfDice matches myPoint, line 37 sets gameStatus to Status::won, and the loop terminates.
 - ◆ If sumOfDice is 7, the program sets gameStatus to Status::lost (line 40), and the loop terminates.
 - ◆ Otherwise, the loop continues executing.

```
33 // while game is not complete
34 while (Status::keepRolling == gameStatus) { // not won or lost
35     // roll dice again and determine game status
36     if (const int sumOfDice{rollDice()}; sumOfDice == myPoint) {
37         gameStatus = Status::won;
38     }
39     else if (sumOfDice == 7) { // lose by rolling 7 before point
40         gameStatus = Status::lost;
41     }
42 }
```

Case Study: Game of Chance; Introducing Scoped enums (Cont.)

- Displaying Whether the Player Won or Lost
 - When the preceding loop terminates, the program proceeds to the following if...else statement, which prints
 - ◆ "Player wins" if gameStatus is Status::won or
 - ◆ "Player loses" if gameStatus is Status::lost.

```
43  
44  
45  // display won or lost message  
46  if (Status::won == gameStatus) {  
47     cout << "Player wins\n";  
48 }  
49  else {  
50     cout << "Player loses\n";  
51 }  
52
```

Player rolled 2 + 5 = 7
Player wins

Player rolled 3 + 3 = 6
Point is 6
Player rolled 5 + 3 = 8
Player rolled 4 + 5 = 9
Player rolled 2 + 1 = 3
Player rolled 1 + 5 = 6
Player wins

Player rolled 6 + 6 = 12
Player loses

Player rolled 1 + 3 = 4
Point is 4
Player rolled 4 + 6 = 10
Player rolled 2 + 4 = 6
Player rolled 6 + 4 = 10
Player rolled 2 + 3 = 5
Player rolled 2 + 4 = 6
Player rolled 1 + 1 = 2
Player rolled 4 + 4 = 8
Player rolled 4 + 3 = 7
Player loses

Case Study: Game of Chance; Introducing Scoped enums (Cont.)

- Function rollDice

- Function rollDice rolls two dice (lines 60–61), calculates their sum (line 62), prints their faces and sum (line 65), and returns the sum (line 67).

```
53 // roll dice, calculate sum and display results
54 int rollDice() {
55     // set up random-number generation
56     static random_device rd; // used to seed the default random engine
57     static default_random_engine engine{rd()}; // rd() produces a seed
58     static uniform_int_distribution randomDie{1, 6};
59
60     const int die1{randomDie(engine)}; // first die roll
61     const int die2{randomDie(engine)}; // second die roll
62     const int sum{die1 + die2}; // compute sum of die values
63
64     // display results of this roll
65     cout << format("Player rolled {} + {} = {}\n", die1, die2, sum);
66
67     return sum;
68 }
```

Case Study: Game of Chance; Introducing Scoped enums (Cont.)

- Function `rollDice`

- Generally, each program that uses random numbers creates one random-number generator engine, seeds it, then uses it throughout the program.
- In this program, only function `rollDice` needs access to the random-number generator, so lines 56–58 define the objects `random_device`, `default_random_engine` and `uniform_int_distribution` as static local variables.
 - ◆ Declaring the objects in lines 56–58 static ensures they are created only the first time `rollDice` is called.
 - ◆ They are then reused in subsequent `rollDice` calls.
 - ◆ Unlike other local variables, which exist only until a function call terminates, static local retain their values between function calls.
- The expression `rd()` in line 57 gets a nondeterministic random integer from the `random_device` object and uses it to seed the `default_random_engine`, enabling the program to produce

```
56 static random_device rd; // used to seed the default random engine
57 static default_random_engine engine{rd()}; // rd() produces a seed
58 static uniform_int_distribution randomDie{1, 6};
```


Case Study: Game of Chance; Introducing Scoped enums (Cont.)

- Additional Notes Regarding Scoped enums
 - Qualifying an enum class's constant with its type name and :: explicitly identifies the constant as being in that type's scope.
 - ◆ If another enum class contains the same identifier, it's always clear which constant is being used because the type name and :: are required.
 - ◆ In general, use unique enum constant values to help prevent hard-to-find logic errors.
 - The following scoped enumeration creates the user-defined enum class type Month with enumeration constants representing the months of the year:

```
enum class Month {jan = 1, feb, mar, apr, may, jun,  
jul, aug, sep, oct, nov, dec};
```
 - Any constant can be assigned an integer value in the enum class definition.
 - ◆ Subsequent constants have a value 1 higher than the preceding constant until the next explicit setting.
 - ◆ The first value in the preceding enumeration is explicitly set to 1, so the remaining values increment from 1, resulting in the values 1 through 12.
 - ◆ You may assign the same value to multiple constants.

Case Study: Game of Chance; Introducing Scoped enums.

- C++20: using enum Declaration

- If the type of an enum class's constants is obvious based on the context in which they're used—such as in our craps example—C++20's using enum declaration allows you to reference an enum class's constants without the type name and scope-resolution operator (::).
- For example, adding the following statement after the enum class declaration

```
using enum Status;
```

- ◆ This would allow the rest of the program to use `keepRolling`, `won` and `lost`, rather than `Status::keepRolling`, `Status::won` and `Status::lost`, respectively.
 - You may also use an individual enum class constant with a declaration of the form
- ```
using Status::keepRolling;
```
- ◆ This would allow your code to use `keepRolling` without the `Status::` qualifier.
  - ◆ Generally, such using declarations should be placed **inside the block** that uses them.

# Function-Call Stack and Activation Records

- To understand how C++ performs function calls, we first need to consider a data structure (i.e., collection of related data items) known as a **stack**.
- Think of a stack as analogous to a pile of dishes. When a dish is placed on the pile, it's placed at the top—referred to as **pushing** the dish onto the stack.
- Similarly, when a dish is removed from the pile, it's removed from the top—referred to as **popping** the dish off the stack.
- Stacks are known as **last-in, first-out (LIFO)** data structures—the last item pushed (inserted) on the stack is the first item popped (removed) from the stack.

# Function-Call Stack and Activation Records (Cont.)

- Function-Call Stack

- One of the most important mechanisms for computer science students to understand is the [function-call stack](#).
  - ◆ Sometimes referred to as the program-execution stack.
- This data structure—working “behind the scenes”—supports the function call/return mechanism.
  - ◆ It also supports the creation, maintenance and destruction of each called function’s local variables.
- As we’ll see in this section’s diagrams, last-in, first-out (LIFO) behavior is precisely what a function needs to return to the function that called it.

# Function-Call Stack and Activation Records (Cont.)

- Stack Frames

- As each function is called, it may, in turn, call other functions, which may, in turn, call other functions—all before any of the functions return.
  - ◆ Each function eventually must return control to the function that called it.
  - ◆ So, somehow, the system must keep track of the return addresses that each function needs to return control to the function that called it.
- The function-call stack is the perfect data structure for handling this information.
  - ◆ Each time a function calls another function, an entry is pushed onto the stack.
  - ◆ This entry—called a [stack frame](#) or an [activation record](#)—contains the return address that the called function needs to return to the calling function.
  - ◆ It also contains some additional information we'll soon discuss.
  - ◆ If the called function returns instead of calling another function before returning, the stack frame for the function call is popped, and control transfers to the return address in the popped stack frame.

# Function-Call Stack and Activation Records (Cont.)

- Stack Frames

- The beauty of the call stack is that each called function always finds the information it needs to return to its caller at the top of the call stack.
- If a function makes calls another function, a stack frame for the new function call is simply pushed onto the call stack.
  - ◆ Thus, the return address required by the newly called function to return to its caller is now at the top.

# Function-Call Stack and Activation Records (Cont.)

- Local Variables and Stack Frames

- The stack frames have another important responsibility.
- Most functions have **local variables**— parameters and any local variables the function declares.
  - ◆ Non-static local variables need to exist while a function is executing.
  - ◆ They need to remain active if the function makes calls to other functions.
  - ◆ But when a called function returns to its caller, the called function's non-static local variables need to “go away.”
- The called function's stack frame is a perfect place to reserve the memory for the called function's non-static local variables.
  - ◆ That stack frame exists as long as the called function is active.
  - ◆ When that function returns—and no longer needs its non-static local variables—its stack frame is popped from the stack, and those non-static local variables no longer exist.

# Function-Call Stack and Activation Records (Cont.)

- Stack Overflow

- Of course, the amount of memory in a computer is finite, so only a certain amount of memory can be used to store activation records on the function-call stack.
- If more function calls occur than can have their activation records stored on the function-call stack, a fatal error known as **stack overflow** occurs.



# Function-Call Stack and Activation Records (Cont.)

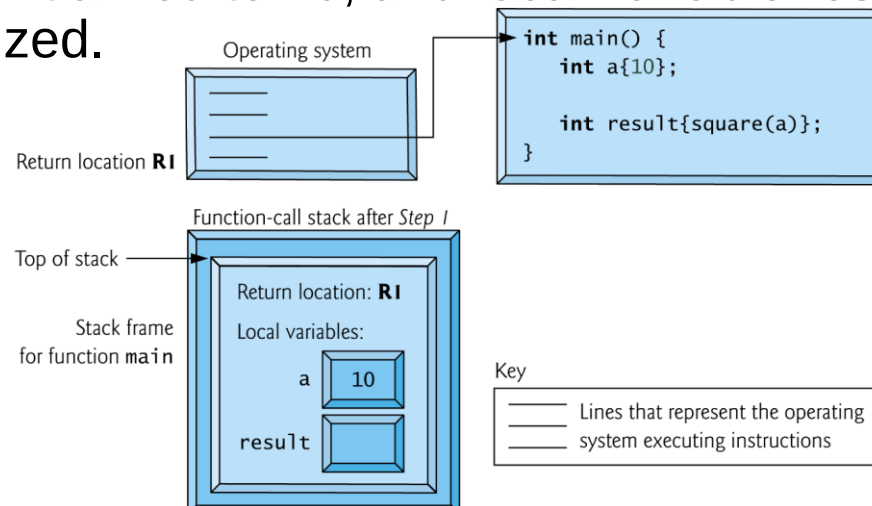
- Function-Call Stack in Action

- Now let's consider how the call stack supports the operation of a square function called by main (lines 8–12).

```
1 // square function used to demonstrate the function
2 // call stack and activation records.
3 #include <iostream>
4 using namespace std;
5
6 int square(int); // prototype for function square
7
8 int main() {
9 int a{10}; // value to square (local variable in main)
10
11 int result{square(a)}; // calculate s squared and store in result
12 }
13
14 // returns the square of an integer
15 int square(int x) { // x is a local variable
16 return x * x; // calculate square and return result
17 }
```

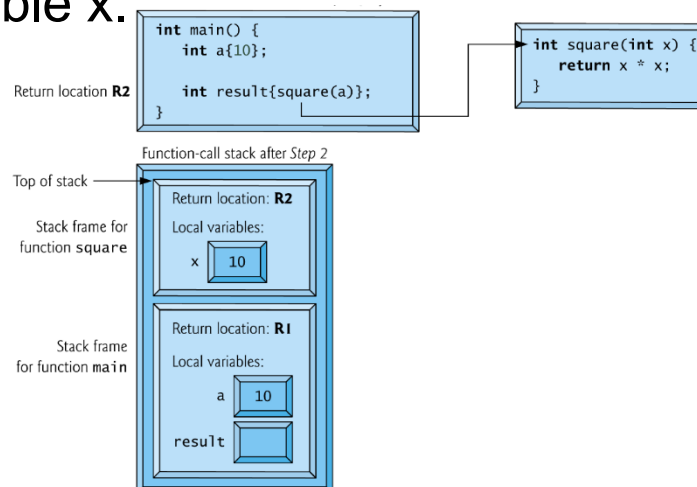
# Function-Call Stack and Activation Records (Cont.)

- Step 1: Operating System Invokes main to Execute the Application
  - First, the operating system calls main—this pushes a stack frame onto the stack.
  - The stack frame tells main how to return to the operating system and contains the space for main's local variable a, which is initialized to 10, and local variable result, which is not yet initialized.



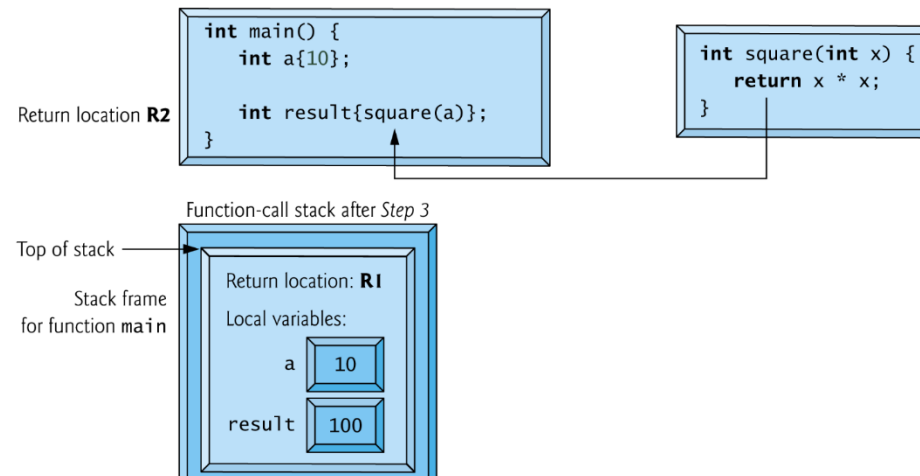
# Function-Call Stack and Activation Records (Cont.)

- Step 2: main Invokes Function square to Perform the Calculation
  - Before returning to the operating system, main calls function square in line 11. This pushes a stack frame for square (lines 15–17) onto the function-call stack.
  - This stack frame contains the return address that square needs to return to main (that is, R2) and the memory for square's local variable x.



# Function-Call Stack and Activation Records (Cont.)

- Step 3: square Returns Its Result to main
  - After square calculates the square of its argument, it needs to return to main—and no longer needs the memory for its variable x.
  - So the stack is popped, giving square the return location in main (that is, R2) and losing square's local variable.



# Function-Call Stack and Activation Records (Cont.)

- Step 4: main Returns Its Result to OS
  - Reaching the closing right brace of main pops its stack frame from the stack. This gives main the address it needs to return to the operating system (R1 in the preceding diagram). At this point, main's local variables a and result are no longer available.

# Function-Call Stack and Activation Records.

- Data Structures

- You've now seen how valuable the stack data structure is in supporting program execution.
- Data structures have many important applications in computer science.
- We already introduced C++'s array and vector data structures and continue discussing those and more in Chapter 13, Data Structures: Standard Library Containers and Iterators.

# Inline Functions

- Implementing a program as a set of functions is good from a software-engineering standpoint, but function calls involve execution-time overhead.
- C++ provides **inline functions** to help reduce function-call overhead.
  - Placing inline before a function's return type in the function definition advises the compiler to generate a copy of the function's body code in every place where the function is called (when appropriate) to avoid a function call.
  - This can make the program larger.
  - The compiler can ignore the inline qualifier.
  - Reusable inline functions are typically placed in headers so that their definitions can be inlined in each source file that uses them.

# Inline Functions (Cont.)

- If you change an inline function's definition, you must recompile any code that calls that function.
  - Compilers can inline code that you do not explicitly declare inline.
  - However, the C++ Core Guidelines recommend declaring “small and time-critical” functions inline.



# Inline Functions.

- The next program uses the inline function cube (lines 8–10) to calculate the volume of a cube.

```
1 // inline function that calculates the volume of a cube.
2 #include <iostream>
3 using namespace std;
4
5 // Definition of inline function cube. Definition of function appears
6 // before function is called, so a function prototype is not required.
7 // First line of function definition also acts as the prototype.
8 inline double cube(double side) {
9 return side * side * side; // calculate cube
10 }
11
12 int main() {
13 double sideValue; // stores value entered by user
14 cout << "Enter the side length of your cube: ";
15 cin >> sideValue; // read value from user
16
17 // calculate cube of sideValue and display result
18 cout << "Volume of cube with side "
19 << sideValue << " is " << cube(sideValue) << '\n';
20 }
```

Enter the side length of your cube: 3.5  
Volume of cube with side 3.5 is 42.875

# References and Reference Parameters

- Two ways to pass arguments to functions in many programming languages are **pass-by-value** and **pass-by-reference**.
  - With pass-by-value, a **copy** of the argument's value is made and passed to the called function.
    - ◆ Changes to the copy do not affect the original variable's value in the caller.
    - ◆ This prevents the accidental side effects that so greatly hinder the development of correct and reliable software systems.
    - ◆ So far, each argument in this book has been passed by value.
    - ◆ One disadvantage of pass-by-value for large data items is that copying data can take considerable execution time and memory space.

# References and Reference Parameters (Cont.)

- Reference Parameters

- This section introduces reference parameters—one of two mechanisms to perform pass-by-reference.
  - ◆ When a variable is passed by reference, the caller gives the called function the ability to access that variable in the caller directly and to modify the variable.
  - ◆ Pass-by-reference is good for performance reasons because it can **eliminate the pass-by-value overhead** of copying large amounts of data.
  - ◆ But pass-by-reference can **weaken security**—the called function can corrupt the caller's data.
- We'll show how to achieve the performance advantage of pass-by-reference while simultaneously achieving the software-engineering advantage of protecting the caller's data from corruption.

# References and Reference Parameters (Cont.)

- Reference Parameters

- A reference parameter is an **alias** for its corresponding argument in a function call.
- To indicate that a function parameter is passed by reference, place an **ampersand (&)** after the parameter's type in the function prototype.
- Use the same convention when listing the parameter's type in the function header.
- For example, the parameter declaration  
`int& number`  
when reading from right to left is pronounced, "number is a reference to an int."
- As always, the function prototype and header must agree.
- In the function call, simply mention a variable's name to pass it by reference.
- In the called function's body, the reference parameter (e.g., `number`) refers to the original variable in the caller, which can be modified directly by the called function.

# References and Reference Parameters (Cont.)

- Passing Arguments by Value and by Reference
  - The next program compares pass-by-value and pass-by-reference with reference parameters.
  - The argument “styles” in the calls to `squareByValue` and `squareByReference` are identical—both variables are simply mentioned by name.
  - The compiler checks the function prototypes and definitions to determine whether to use pass-by-value or pass-by-reference.

# References and Parameters (Co

```
24 // squareByValue multiplies number by itself, stores the
25 // result in number and returns the new value of number
26 int squareByValue(int number) {
27 return number *= number; // caller's argument not modified
28 }
29
30 // squareByReference multiplies numberRef by itself and stores the result
31 // in the variable to which numberRef refers in function main
32 void squareByReference(int& numberRef) {
33 numberRef *= numberRef; // caller's argument modified
34 }
```

- Passing Arguments by Value and by Reference

```
1 // Passing arguments by value and by reference.
2 #include <iostream>
3 using namespace std;
4
5 int squareByValue(int number); // prototype (for value pass)
6 void squareByReference(int& numberRef); // prototype (for reference pass)
7
8 int main() {
9 int x{2}; // value to square using squareByValue
10 int z{4}; // value to square using squareByReference
11
12 // demonstrate squareByValue
13 cout << "x = " << x << " before squareByValue\n";
14 cout << "Value returned by squareByValue: "
15 << squareByValue(x) << '\n';
16 cout << "x = " << x << " after squareByValue\n\n";
17
18 // demonstrate squareByReference
19 cout << "z = " << z << " before squareByReference\n";
20 squareByReference(z);
21 cout << "z = " << z << " after squareByReference\n";
22 }
```

x = 2 before squareByValue  
Value returned by squareByValue: 4  
x = 2 after squareByValue

z = 4 before squareByReference  
z = 16 after squareByReference

# References and Reference Parameters (Cont.)

- References as Aliases within a Function

- References can also be used as aliases for other variables within a function. For example, the code

```
int count{1}; // declare integer variable count
int& cRef{count}; // create cRef as an alias for count
++cRef; // increment count (using its alias cRef)
increments count by using its alias cRef.
```

- Reference variables must be **initialized in their declarations** and cannot be reassigned as aliases for other variables.
  - ◆ In this sense, references are constant.

# References and Reference Parameters (Cont.)

- References as Aliases within a Function
  - All operations performed on the alias (i.e., the reference) are actually performed on the original variable.
    - ◆ The alias is simply another name for the original variable.
  - Unless it's a reference to a constant, a reference's initializer must be an lvalue—something that can appear on the left side of an assignment, like a variable name.
  - A reference may not be initialized with a constant or rvalue expression—that is, something that may only appear on the right side of an assignment, such as the result of a calculation.



# References and Reference Parameters (Cont.)

- const References

- To specify that a reference parameter should not be allowed to modify the corresponding argument in the caller, place the **const qualifier before the type name** in the parameter's declaration. For example, consider a `displayName` function:

```
void displayName(std::string name) {
 std::cout << name << '\n';
}
```

- ◆ When called, it receives a copy of its string argument.
- ◆ Since string objects can be large, this copy operation could degrade an application's performance.
- ◆ For this reason, string objects (and objects in general) should be passed to functions by reference.

# References and Reference Parameters (Cont.)

- **const References**

- Also, the `displayName` function does not need to modify its argument. So, following the principle of least privilege, we'd declare the parameter as

```
const std::string& name
```

- ◆ Reading this from right to left, the `name` parameter is a reference to a string that is constant. We get the performance of passing the string by reference.
- ◆ Also, `displayName` treats the argument as a constant, so `displayName` cannot modify the value in the caller.

# References and Reference Parameters.

- Returning a Reference to a Local Variable Is Dangerous

- When returning a reference to a local non-static variable, the reference refers to a variable that's discarded when the function returns.
- Attempting to access such a variable yields **undefined behavior**, often crashing the program or corrupting data.
- References to undefined variables are called **dangling references**.
  - ◆ This is a logic error for which compilers often issue a warning.
  - ◆ Software-engineering teams often have policies requiring that code must compile without warnings before it can be deployed.
  - ◆ You can enforce this for your team by using your compiler's option that causes it to treat warnings as errors.

# Default Arguments

- It's common for a program to invoke a function from several places with the same argument value for a particular parameter.
- In such cases, you can specify a default argument for the corresponding parameter.
- When a program omits an argument for a parameter with a default argument in a function call, the compiler rewrites the function call, inserting the default value of that argument.

# Default Arguments (Cont.)

- boxVolume Function with Default Arguments

- The next program demonstrates using default arguments to calculate a box's volume.
- The function prototype for boxVolume (line 6) specifies that all three parameters have default values of 1 by placing = 1 to the right of each parameter.

```
1 // Using default arguments.
2 #include <iostream>
3 using namespace std;
4
5 // function prototype that specifies default arguments
6 int boxVolume(int length = 1, int width = 1, int height = 1);
7
26 // function boxVolume calculates the volume of a box
27 int boxVolume(int length, int width, int height) {
28 return length * width * height;
29 }
```

# Default Arguments (Cont.)

- boxVolume Function with Default Arguments

- The first call to boxVolume (line 10) specifies no arguments, thus using all three default values of 1.
- The second call (line 14) passes only a length argument, thus using default values of 1 for the width and height arguments.
- The third call (line 18) passes arguments for only length and width, thus using a default value of 1 for the height argument.
- The last call (line 22) passes arguments for length, width and height, thus using no default values.

```
10 cout << "The default box volume is: " << boxVolume();
11
12 // specify length; default width and height
13 cout << "\n\nThe volume of a box with length 10,\n"
14 << "width 1 and height 1 is: " << boxVolume(10);
15
16 // specify length and width; default height
17 cout << "\n\nThe volume of a box with length 10,\n"
18 << "width 5 and height 1 is: " << boxVolume(10, 5);
19
20 // specify all arguments
21 cout << "\n\nThe volume of a box with length 10,\n"
22 << "width 5 and height 2 is: " << boxVolume(10, 5, 2)
23 << '\n';
```

The default box volume is: 1

The volume of a box with length 10,  
width 1 and height 1 is: 10

The volume of a box with length 10,  
width 5 and height 1 is: 50

The volume of a box with length 10,  
width 5 and height 2 is: 100

# Default Arguments (Cont.)

- boxVolume Function with Default Arguments
  - Any arguments passed to the function explicitly are assigned to the function's parameters from left to right.
    - ◆ Therefore, when boxVolume receives one argument, the function assigns the value of that argument to its length parameter (i.e., the leftmost parameter in the parameter list).
    - ◆ When boxVolume receives two arguments, the function assigns those arguments' values to the length and width parameters in that order.
    - ◆ Finally, when boxVolume receives all three arguments, the function assigns the values of those arguments to the length, width and height parameters, respectively.

# Default Arguments (Cont.)

- Notes Regarding Default Arguments

- Default arguments must be the **rightmost (trailing)** arguments in a function's parameter list.
  - ◆ When calling a function with two or more default arguments, if an omitted argument is not the rightmost argument, then all arguments to the right of that argument also must be omitted.
- Default arguments must be specified with the **first occurrence of the function name**—typically, in the function prototype.
  - ◆ If the function prototype is omitted because the function definition also serves as its prototype, the default arguments should be specified in the function header.



# Default Arguments.

- Notes Regarding Default Arguments

- Default values can be **any expression**, including constants, global variables or function calls.
- Default arguments also can be used **with inline functions**.
- Using default arguments can simplify writing function calls, but some programmers feel that **explicitly specifying all arguments is clearer**.

# Function Overloading

- C++ allows functions of the same name to be defined, provided that they have different signatures.
- This is called function overloading: The C++ compiler selects the proper function to call by examining the number, types and order of the arguments in the call.
- Function overloading is used to create several functions of the same name that **perform similar tasks** but on data of different types.
- For example, many functions in the **math library** are overloaded for different numeric types—the C++ standard requires float, double and long double versions of each math library function.
- Overloading functions that perform closely related tasks can make programs clearer.

# Function Overloading (Cont.)

- Overloaded square Functions

- The next program uses overloaded square functions to calculate the square of an int (lines 6–9) and the square of a double (lines 12–15).

```
1 // Overloaded square functions.
2 #include <iostream>
3 using namespace std;
4
5 // function square for int values
6 int square(int x) {
7 cout << "square of integer " << x << " is ";
8 return x * x;
9 }
10
11 // function square for double values
12 double square(double y) {
13 cout << "square of double " << y << " is ";
14 return y * y;
15 }
```

# Function Overloading (Cont.)

- Overloaded square Functions

- Line 18 invokes the int version by passing the literal value 7.
  - ◆ C++ treats whole-number literal values as ints.
- Similarly, line 20 invokes the double version by passing the literal value 7.5, which C++ treats as a double.
- In each case, the compiler chooses the proper function to call based on the type of the argument.

```
17 int main() {
18 cout << square(7); // calls int version
19 cout << '\n';
20 cout << square(7.5); // calls double version
21 cout << '\n';
22 }
```

square of integer 7 is 49  
square of double 7.5 is 56.25

# Function Overloading (Cont.)

- How the Compiler Differentiates Among Overloaded Functions
  - Overloaded functions are distinguished by their **signatures**. A signature is a combination of a **function's name and its parameter types (in order)**.
  - Type-safe linkage ensures that the proper function is called and that the arguments' types conform to the parameters' types.
  - To enable type-safe linkage, the compiler internally encodes each function identifier with the types of its parameters—a process referred to as name mangling.
    - ◆ These encodings vary by compiler, so everything that will be linked to create an executable for a given platform must be compiled using the same compiler for that platform.

# Function Overloading (Cont.)

- How the Compiler Differentiates Among Overloaded Functions
  - For GNU C++, each **mangled name** (other than main) begins with an underscore (`_`) followed by the letter Z, a number and the function name.
  - The number that follows Z specifies how many characters are in the function's name. For example, function `square` has 6 characters in its name, so its mangled name is prefixed with `_Z6`.
  - Following the function name is an encoding of its parameter list.

# Function Overloading (Cont.)

- How the Compiler Differentiates Among Overloaded Functions
  - For function square that receives an int (line 4), i represents int, as shown in the output's first line.
  - For function square that receives a double (line 9), d represents double, as shown in the output's second line.

```
1 // Name mangling to enable type-safe linkage.
2
3 // function square for int values
4 int square(int x) {
5 return x * x;
6 }
7
8 // function square for double values
9 double square(double y) {
10 return y * y;
11 }
```

|                                                    |
|----------------------------------------------------|
| <code>_Z6squarei</code><br><code>_Z6squarel</code> |
|----------------------------------------------------|

# Function Overloading (Cont.)

- How the Compiler Differentiates Among Overloaded Functions

- For function nothing1 (line 15), i represents an int, f represents a float, c represents a char, and Ri represents an int& (i.e., a reference to an int), as shown in the output's third line.
- For function nothing2 (line 19), c represents a char, i represents an int, Rf represents a float& and Rd represents a double&.

```
13 // function that receives arguments of types
14 // int, float, char and int&
15 void nothing1(int a, float b, char c, int& d) { }
16
17 // function that receives arguments of types
18 // char, int, float& and double&
19 int nothing2(char a, int b, float& c, double& d) {
20 return 0;
21 }
22
23 int main() { }
```

\_Z8nothing1ifcRi  
\_Z8nothing2ciRfRd  
main



# Function Overloading (Cont.)

- How the Compiler Differentiates Among Overloaded Functions
  - A function with default arguments omitted might be called identically to another overloaded function; this is a compilation error.
    - ◆ For example, having a program that contains both a function that explicitly takes no arguments and a function of the same name that contains all default arguments results in a compilation error when an attempt is made to use that function name in a call passing no arguments.
    - ◆ The compiler cannot unambiguously determine which version of the function to choose.

# Function Overloading.

- Overloaded Operators

- Chapter 11 discusses how to overload operators to define how they should operate on objects of user-defined data types.
- In fact, we've been using many overloaded operators to this point, including the stream insertion << and the stream extraction >> operators.
  - ◆ These are overloaded for all the fundamental types.
  - ◆ We say more about overloading << and >> to be able to handle objects of user-defined types in Chapter 11.

# Function Templates

- Overloaded functions typically perform similar operations on different data types.
- If the program logic and operations are identical for each data type, overloading may be performed more compactly and conveniently with function templates.
  - You write a single function template definition, and C++ automatically generates separate function template specializations (also called template instantiations) to appropriately handle each type of call for the provided argument types.
  - Thus, a single function template defines a family of overloaded functions.

# Function Templates (Cont.)

- Programming with templates is also known as generic programming.
  - In this section, we define a custom function template.
  - In subsequent chapters, you'll use many C++ standard library templates, and in Chapter 15, we'll discuss defining custom templates in detail.

# Function Templates (Cont.)

- maximum Function Template
  - The next program defines a maximum function template that returns the largest of three values.

```
1 // Function template maximum header.
2 template <typename T> // or template <class T>
3 T maximum(T value1, T value2, T value3) {
4 T maximumValue{value1}; // assume value1 is maximum
5
6 // determine whether value2 is greater than maximumValue
7 if (value2 > maximumValue) {
8 maximumValue = value2;
9 }
10
11 // determine whether value3 is greater than maximumValue
12 if (value3 > maximumValue) {
13 maximumValue = value3;
14 }
15
16 return maximumValue;
17 }
```

# Function Templates (Cont.)

- maximum Function Template

- All function template definitions begin with the template keyword (line 2) followed by a template parameter list enclosed in angle brackets (< and >).
- Every parameter in the template parameter list is preceded by keyword typename or keyword class (they are synonyms in this context).

```
2 template <typename T> // or template <class T>
3 T maximum(T value1, T value2, T value3) {
4 T maximumValue{value1}; // assume value1 is maximum
5
6 // determine whether value2 is greater than maximumValue
7 if (value2 > maximumValue) {
8 maximumValue = value2;
9 }
10
11 // determine whether value3 is greater than maximumValue
12 if (value3 > maximumValue) {
13 maximumValue = value3;
14 }
15
16 return maximumValue;
17 }
```

# Function Templates (Cont.)

- maximum Function Template

- The type parameters are placeholders for fundamental types or user-defined types.
  - ◆ These placeholders—in this case, T—are used to specify the types of the function's parameters (line 3), to specify the function's return type (line 3) and to declare variables within the body of the function definition (line 4).
- A function template is defined like any other function but uses the type parameters as placeholders for actual data types.

```
1 // Function template maximum header.
2 template <typename T> // or template <class T>
3 T maximum(T value1, T value2, T value3) {
4 T maximumValue{value1}; // assume value1 is maximum
```

# Function Templates (Cont.)

- maximum Function Template

- maximum's one type parameter T (line 2) is a placeholder for the type of data the function processes.
- Type parameter names in a template parameter list must be unique.
- When the compiler encounters a call to maximum, it substitutes the argument types in the call for T throughout the template definition, creating a complete function template specialization that determines the maximum of three values of the specified type.
- The values must have the same type because we use only one type parameter.

```
1 // Function template maximum header.
2 template <typename T> // or template <class T>
3 T maximum(T value1, T value2, T value3) {
4 T maximumValue{value1}; // assume value1 is maximum
```



# Function Templates.

- Using Function Template maximum

```
1 // Function template maximum test program.
2 #include <iostream>
3 #include "maximum.h" // include definition of function template maximum
4 using namespace std;
5
6 int main() {
7 // demonstrate maximum with int values
8 cout << "Input three integer values: ";
9 int int1, int2, int3;
10 cin >> int1 >> int2 >> int3;
11
12 // invoke int version of maximum
13 cout << "The maximum integer value is: "
14 << maximum(int1, int2, int3);
15
16 // demonstrate maximum with double values
17 cout << "\n\nInput three double values: ";
18 double double1, double2, double3;
19 cin >> double1 >> double2 >> double3;
20
21 // invoke double version of maximum
22 cout << "The maximum double value is: "
23 << maximum(double1, double2, double3);
24
25 // demonstrate maximum with char values
26 cout << "\n\nInput three characters: ";
27 char char1, char2, char3;
28 cin >> char1 >> char2 >> char3;
29
30 // invoke char version of maximum
31 cout << "The maximum character value is: "
32 << maximum(char1, char2, char3) << '\n';
33 }
```

Input three integer values: 1 2 3  
The maximum integer value is: 3

Input three double values: 3.3 2.2 1.1  
The maximum double value is: 3.3

Input three characters: A C B  
The maximum character value is: C

# Scope Rules

- The portion of a program where an identifier can be used is known as its scope. For example, when we declare a local variable in a block, it can be referenced only
  - from the point of its declaration in that block to the end of that block and
  - in nested blocks that appear within that block after the variable's declaration.
- This section discusses block scope and global namespace scope.
  - Parameter names in function prototypes have function parameter scope and are known only in the prototype in which they appear.
  - Later we'll see other scopes, including class scope in Chapter 9 and function scope and namespace scope in Chapter 20.

# Scope Rules (Cont.)

- Block Scope

- Identifiers declared in a block have **block scope**, which begins at the identifier's declaration and ends at the block's terminating right brace ({}).
  - ◆ Local variables have block scope, as do function parameters.
  - ◆ Any block can contain variable declarations.
    - In nested blocks, if an identifier in an outer block has the same name as an identifier in an inner block, the one in the outer block is "**hidden**" while the inner block is executing.
    - The inner block "sees" its own local variable's value and not that of the enclosing block's identically named variable.
    - It is typically a logic error to accidentally use the same name for an identifier in an inner block that's used for an identifier in an outer block when, in fact, you want the identifier in the outer block to be visible in the inner block.
    - Avoid variable names in inner scopes that hide names in outer scopes. Most compilers will warn you about this.

# Scope Rules (Cont.)

- Block Scope

- In the next program, local variables may also be declared `static`.
  - ◆ Such variables also have block scope.
- Unlike other local variables, a static local variable retains its value when the function returns to its caller.
  - ◆ The next time the function is called, the static local variable contains the value it had when the function executed last time—not reinitialized.
  - ◆ The following statement declares a static local variable `count` and initializes it to 1:  

```
static int count{1};
```
  - ◆ By default, static local variables of numeric types are initialized to zero—though explicit initialization is preferred.
  - ◆ Default initialization of non-fundamental-type variables depends on the type. For example, a string's default value is the empty string `""`.

# Scope Rules (Cont.)

- Global Namespace Scope

- An identifier declared **outside any function or class** has **global namespace scope**.
- Such an identifier is “known” to all functions after its declaration in the source-code file.
- Function definitions, function prototypes placed outside a function, class definitions and global variables all have global namespace scope.
- Global variables are created by placing variable declarations outside any class or function definition.
  - ◆ Such variables retain their values throughout a program’s execution.

# Scope Rules (Cont.)

- Global Namespace Scope

- Declaring a variable as global rather than local allows unintended side effects to occur when a function that does not need access to the variable accidentally or maliciously modifies it.
- Except for truly global resources, like `cin` and `cout`, avoid global variables.
  - ◆ This is an example of the [principle of least privilege](#), which is fundamental to good software engineering.
  - ◆ It states that code should be granted only the amount of privilege and access needed to accomplish its designated task, but no more.
  - ◆ An example is the scope of a local variable, which should not be visible when it's not needed.
    - A local variable is created when the function is called, used by that function while it executes, then goes away when the function returns.
    - The principle of least privilege makes your programs more **robust** by preventing code from accidentally (or maliciously) modifying variable values that should not be accessible.
    - It also makes your programs easier to read and maintain.

# Scope Rules (Cont.)

- Global Namespace Scope

- In general, variables should be declared in the narrowest scope in which they need to be accessed.
- Variables used only in a particular function should be declared as local variables in that function rather than as global variables.

# Scope Rules (Cont.)

- Scope Demonstration

- The next program demonstrates scoping issues with global variables, local variables and static local variables.
- Line 9 declares and initializes global variable `x` to 1. We'll show that this global variable is hidden in any block (or function) that declares a variable named `x`.



# Scope Rules (Cont.)

```

1 // Scoping example.
2 #include <iostream>
3 using namespace std;
4
5 void useLocal(); // function prototype
6 void useStaticLocal(); // function prototype
7 void useGlobal(); // function prototype
8
9 int x{1}; // global variable
10
11 int main() {
12 cout << "global x in main is " << x << '\n';
13
14 const int x{5}; // local variable to main
15
16 cout << "local x in main's outer scope is " << x << '\n';
17
18 { // block starts a new scope
19 const int x{7}; // hides both x in outer scope and global x
20
21 cout << "local x in main's inner scope is " << x << '\n';
22 }
23
24 cout << "local x in main's outer scope is " << x << '\n';
25
26 useLocal(); // useLocal has local x
27 useStaticLocal(); // useStaticLocal has static local x
28 useGlobal(); // useGlobal uses global x
29 useLocal(); // useLocal reinitializes its local x
30 useStaticLocal(); // static local x retains its prior value
31 useGlobal(); // global x also retains its prior value
32
33 cout << "\nlocal x in main is " << x << '\n';
34 } // end of main

```

global x in main is 1  
 local x in main's outer scope is 5  
 local x in main's inner scope is 7  
 local x in main's outer scope is 5

local x is 25 on entering useLocal  
 local x is 26 on exiting useLocal

local static x is 50 on entering useStaticLocal  
 local static x is 51 on exiting useStaticLocal

global x is 1 on entering useGlobal  
 global x is 10 on exiting useGlobal

local x is 25 on entering useLocal  
 local x is 26 on exiting useLocal

local static x is 51 on entering useStaticLocal  
 local static x is 52 on exiting useStaticLocal

global x is 10 on entering useGlobal  
 global x is 100 on exiting useGlobal

local x in main is 5

# Scope Rules (Cont.)

- Unary Scope Resolution Operator

- C++ provides the **unary scope resolution operator (::)** to access a global variable when a local variable of the same name is in scope.
- The unary scope resolution operator cannot be used to access a local variable of the same name in an outer block.
- A global variable can be accessed directly without the unary scope resolution operator if the name of the global variable is not the same as that of a local variable in scope.

# Scope Rules (Cont.)

- Unary Scope Resolution Operator

- The next program shows the unary scope resolution operator with global and local variables of the same name (lines 5 and 8).
- To emphasize that the local and global versions of variable number are distinct, the program declares one variable int and the other double.

```
1 // Unary scope resolution operator.
2 #include <iostream>
3 using namespace std;
4
5 const int number{7}; // global variable named number
6
7 int main() {
8 const double number{10.5}; // local variable named number
9
10 // display values of local and global variables
11 cout << "Local double value of number = " << number
12 << "\nGlobal int value of number = " << ::number << '\n';
13 }
```

Local double value of number = 10.5  
Global int value of number = 7

# Scope Rules.

- Unary Scope Resolution Operator

- In general, you should avoid using variables of the same name for different purposes in a program. Although this is allowed in various circumstances, it can lead to errors.
- Generally, global variables are discouraged.
  - ◆ If you use one, always use the unary scope resolution operator (::) to refer to it (even if there is no collision with a local-variable name).
  - ◆ This makes it clear that you're accessing a global variable.
  - ◆ It also makes programs easier to modify by reducing the risk of name collisions with nonglobal variables and eliminates logic errors that might occur if a local variable hides the global variable.

# Lnfylun Lhqtomh Wjtz Qarcv: Qjwazkrplm xzz Xndmwwqhlz

- You probably noticed that the last Objectives bullet for this chapter, the last section name in the chapter outline, and the section title above all look like gibberish.
- These are not mistakes! In this section, we continue our Objects-Natural presentation.
- You'll conveniently encrypt and decrypt messages with an object you create of a preexisting class that implements a Vigenère secret key cipher.

# Lnfylun Lhqtomh Wjtz Qarcv: Qjwazkrplm xzz Xndmwwqhlz

- In previous Objects-Natural sections, you created objects of built-in C++ standard library class string and objects of classes from open-source libraries.
- Sometimes you'll use classes built by your organization or team members for internal use or for use in a specific project.
- For this example, we wrote our own Cipher class (in the header "cipher.h") and provided it to you.

# Lnfylun Lhqtomh Wjtz Qarcv: Qjwazkrplm xzz Xndmwwqhlz

- For this Objects Natural section, we show you how easy it is to do encryption and decryption with an object of a pre-existing class.
- In Chapter 9, Custom Classes, you'll build custom classes.
  - We will explain the Vigenère secret key cipher algorithm and gives you the opportunity to study our custom class to see how it implements the cipher.

# Lnfylun Lhqtomh Wjtz Qarcv: Qjwazkrplm xzz Xndmwwqhlz

- Cryptography

- Cryptography has been used for thousands of years and is critically important in today's connected world.
- Every day, cryptography is used behind the scenes to ensure that your Internet-based communications are private and secure.
  - ◆ For example, most websites now use the HTTPS protocol to encrypt and decrypt your web interactions, as we do at [deitel.com](http://deitel.com).



# Lnfylun Lhqtomh Wjtz Qarcv: Qjwazkrplm xzz Xndmwwqhlz

## • Caesar Cipher

- Julius Caesar used a simple substitution cipher to encrypt military communications.
- His technique—known as the Caesar cipher—replaces every letter in a communication with the letter three ahead in the alphabet.
  - ◆ So, A is replaced with D, B with E, C with F, ... X with A, Y with B and Z with C.
- Thus, the plaintext  
Caesar Cipher  
would be encrypted as  
Fdhvdu Flskhu
- The encrypted text is known as the ciphertext.

# Lnfylun Lhqtomh Wjtz Qarcv: Qjwazkrplm xzz Xndmwwqhlz

- Vigenère Cipher

- Simple substitution ciphers like the Caesar cipher are relatively easy to decrypt.
  - ◆ For example, the letter “e” is the most frequently used letter in English.
  - ◆ So, you could study ciphertext encrypted with the Caesar cipher and assume with a high likelihood that the character appearing most frequently is probably an “e.”
- In this example, you’ll use a Vigenère cipher, which is a secret-key substitution cipher.
  - ◆ This cipher is implemented using 26 Caesar ciphers—one for each letter of the alphabet.
  - ◆ A Vigenère cipher uses letters from the plaintext and secret key to look up replacement characters in the various Caesar ciphers.

# Lnfylun Lhqtomh Wjtz Qarcv: Qjwazkrplm xzz Xndmwwqhlz

- Vigenère Cipher.

- For this cipher, the secret key must be composed of letters.
  - ◆ Like passwords, the secret key should not be easy to guess.
- We used the 11 randomly selected characters  
XMWUJBVYHXZ
- There's no limit to the number of characters you can use in your secret key.
  - ◆ However, the person decrypting the ciphertext must know the secret key originally used to create the ciphertext.
  - ◆ Presumably, you'd provide that in advance—possibly in a face-to-face meeting.

# Lnfylun Lhqtomh Wjtz Qarcv: Qjwazkrplm xzz Xndmwwqhlz

- Using Our Cipher Class

- For the next example, you'll use our class , which implements the Vigenère cipher.
- The header (line 2) from the examples folder defines the class.
- You don't need to read and understand the class's code to use its encryption and decryption capabilities.
- You simply create an object of class Cipher then call its encrypt and decrypt member functions to encrypt and decrypt text, respectively.

```
1 // Encrypting and decrypting text with a Vigenère cipher.
2 #include "cipher.h"
3 #include <iostream>
4 #include <string>
5 using namespace std;
6
7 int main() {
```

# Lnfylun Lhqtomh Wjtz Qarcv: Qjwazkrplm xzz Xndmwwqhlz

- Using Our Cipher Class

- Lines 10 and 14 call the getline function call to read all the characters you type until you press Enter.
  - ◆ Here, the characters come from cin and are placed into the string in the second argument.

```
8 string plainText;
9 cout << "Enter the text to encrypt:\n";
10 getline(cin, plainText);
11
12 string secretKey;
13 cout << "\nEnter the secret key:\n";
14 getline(cin, secretKey);
```

# Lnfylun Lhqtomh Wjtz Qarcv: Qjwazkrplm xzz Xndmwwqhlz

```
1 // Encrypting and decrypting text with a Vigenère cipher.
2 #include "cipher.h"
3 #include <iostream>
4 #include <string>
5 using namespace std;
6
7 int main() {
8 string plainText;
9 cout << "Enter the text to encrypt:\n";
10 getline(cin, plainText);
11
12 string secretKey;
13 cout << "\nEnter the secret key:\n";
14 getline(cin, secretKey);
15
16 Cipher cipher;
17
18 // encrypt plainText using secretKey
19 string cipherText{cipher.encrypt(plainText, secretKey)};
20 cout << "\nEncrypted:\n " << cipherText << '\n';
21
22 // decrypt cipherText
23 cout << "\nDecrypted:\n "
24 << cipher.decrypt(cipherText, secretKey) << '\n';
25
26 // decrypt ciphertext entered by the user
27 cout << "\nEnter the ciphertext to decipher:\n";
28 getline(cin, cipherText);
29 cout << "\nDecrypted:\n "
30 << cipher.decrypt(cipherText, secretKey) << '\n';
31 }
```

# Lnfylun Lhqtomh Wjtz Qarcv: Qjwazkrplm xzz Xndmwwqhlz

Enter the text to encrypt:

**Welcome to Modern C++ application development with C++20!**

Enter the secret key:

**XMWUJBVYHXZ**

Encrypted:

Tqhwxnz rv Jnaqnh L++ bknsfbxfeiw eztlinmyahc xdro Z++20!

Decrypted:

Welcome to Modern C++ application development with C++20!

Enter the ciphertext to decipher:

**Lnfylun Lhqtomh Wjtz Qarcv: Qjwazkrplm xzz Xndmwwqhlz**

Decrypted:

Objects Natural Case Study: Encryption and Decryption

# Lnfylun Lhqtomh Wjtz Qarcv: Qjwazkrplm xzz Xndmwwqhlz

- Class Cipher's Member Functions

- The class provides two key member functions:

- ◆ encrypt receives strings representing the plaintext to encrypt and the secret key, uses the Vigenère cipher to encrypt the text, then returns a string containing the ciphertext.
    - ◆ decrypt receives strings representing the ciphertext to decrypt and the secret key, reverses the Vigenère cipher to decrypt the text, then returns a string containing the plaintext.



# Lnfylun Lhqtomh Wjtz Qarcv: Qjwazkrplm xzz Xndmwwqhlz

- Class Cipher's Member Functions

- The program first asks you to enter text to encrypt and a secret key.
  - ◆ Line 16 creates the Cipher object.
  - ◆ Lines 19–20 encrypt the text you entered and display the encrypted text.
  - ◆ Then, lines 23–24 decrypt the text to show you the plaintext string you entered.

```
16 Cipher cipher;
17
18 // encrypt plainText using secretKey
19 string cipherText{cipher.encrypt(plainText, secretKey)};
20 cout << "\nEncrypted:\n " << cipherText << '\n';
21
22 // decrypt cipherText
23 cout << "\nDecrypted:\n "
24 << cipher.decrypt(cipherText, secretKey) << '\n';
```

# Lnfylun Lhqtomh Wjtz Qarcv: Qjwazkrplm xzz Xndmwwqhlz

- Class Cipher's Member Functions

- Though the last Objectives bullet in this chapter, and this section's title look like gibberish, they're each ciphertext that we created with our Cipher class and the secret key XMWUJBVYHXZ
- Lines 27–28 prompt for and input existing ciphertext, then lines 29–30 decrypt the ciphertext and display the original plaintext we encrypted.

```
26 // decrypt ciphertext entered by the user
27 cout << "\nEnter the ciphertext to decipher:\n";
28 getline(cin, cipherText);
29 cout << "\nDecrypted:\n"
30 << cipher.decrypt(cipherText, secretKey) << '\n';
31 }
```

# Lnfylun Lhqtomh Wjtz Qarcv: Qjwazkrplm xzz Xndmwwqhlz

- Class Cipher's Possible Results

Enter the text to encrypt:

**Welcome to Modern C++ application development with C++20!**

Enter the secret key:

**XMWUJBVYHXZ**

Encrypted:

Tqhwxnz rv Jnaqnh L++ bknsfbxfeiw eztlinmyahc xdro Z++20!

Decrypted:

Welcome to Modern C++ application development with C++20!

Enter the ciphertext to decipher:

**Lnfylun Lhqtomh Wjtz Qarcv: Qjwazkrplm xzz Xndmwwqhlz**

Decrypted:

Objects Natural Case Study: Encryption and Decryption